Developer Work Assignment

Backend, Smart Contracts & Integration — Complete implementation guide with 50/50 work distribution

PROJECT	VERSION	DATE	CLASSIFICATION
Agentic Dark Pool	V1 — MVP	March 29, 2026	Internal Only

OVERVIEW

Table of Contents

This document defines the complete work assignment for the Agentic Dark Pool backend implementation, split equally between two developers.

SECTION	TITLE	RELEVANCE
01	Work Split Overview	BOTH
02	Shared Setup & Standards	BOTH
03	Sorrowz — Smart Contracts	
04	Sorrowz — Backend Services	
05	Sorrowz — Frontend Integration	
06	Powerz — Smart Contracts	
07	Powerz — Backend Services	
08	Powerz — Frontend Integration	
09	Shared Interfaces & Data Contracts	BOTH
10	Timeline & Milestones	BOTH

SECTION	TITLE	RELEVANCE
11	Dependency Map & Integration Points	BOTH

SECTION 01

Work Split Overview

The implementation is divided into two domains of equal scope and complexity. Each developer owns their domain end-to-end: smart contracts, backend modules, database tables, and frontend wiring.

S

Sorrowz

Core Trading & Settlement Lead

- DarkPool.sol — main contract
- Escrow.sol — fund management
- SettlementVerifier.sol — ZK verification
- Orders Module — CRUD + validation
- Matching Engine — batch auction algorithm
- Settlement Service — on-chain settlement
- Frontend: Order flow + contract hooks
- DB: orders, settlements, compute_jobs

P

Powerz

Infrastructure & Services Lead

- TokenRegistry.sol — token + GPU types
- ComputeCredit.sol — ERC-20 token
- FeeCollector.sol — fee distribution
- Auth Module — API keys + wallet auth
- Provider Module — registration + reputation
- Market Module — stats + analytics
- WebSocket Server — real-time updates
- Event Indexer — blockchain listener
- Frontend: Market data + provider + real-time
- DB: users, providers, api_keys

Workload Distribution

Smart Contracts

Sorrowz: 3 contracts | Powerz: 3 contracts

Backend Modules

Sorrowz: 3 modules (complex) | Powerz: 5 modules (lighter)

Database Tables

Sorrowz: 3 tables | Powerz: 3 tables

Frontend Wiring

Sorrowz: Order flow + contracts | Powerz: Market + provider + WS

Note on complexity balance: Sorrowz has fewer backend modules but they are significantly more complex (the matching engine alone is the hardest piece in the entire system). Powerz has more modules but they are standard CRUD/infra patterns. This balances to approximately equal effort.

SECTION 02

Shared Setup & Standards

Before either developer begins their assigned work, the following must be established collaboratively.

Monorepo Structure

```
// Recommended project structure darkpool/ ├── packages/ │ ├── contracts/
// Foundry project (both devs) │ │ ├── src/ │ │ │ ├── DarkPool.sol //
Sorrowz │ │ │ ├── Escrow.sol // Sorrowz │ │ │ ├── SettlementVerifier.sol //
Sorrowz │ │ │ ├── TokenRegistry.sol // Powerz │ │ │ ├── ComputeCredit.sol
// Powerz │ │ │ └── FeeCollector.sol // Powerz │ │ ├── test/ │ │ ├──
script/ │ │ └── foundry.toml │ │ │ ├── backend/ // NestJS project (both
devs) │ │ ├── src/ │ │ │ ├── orders/ // Sorrowz │ │ │ ├── matching/ //
Sorrowz │ │ │ ├── settlement/ // Sorrowz │ │ │ ├── auth/ // Powerz │ │ │
├── providers/ // Powerz │ │ │ ├── market/ // Powerz │ │ │ ├── websocket/
// Powerz │ │ │ ├── indexer/ // Powerz │ │ │ └── shared/ // Both (types,
utils, config) │ │ ├── prisma/ │ │ └── package.json │ │ │ └── frontend/ //
Existing Vite + React app │ ├── src/ │ │ ├── hooks/ │ │ │ ├── useOrders.ts
// Sorrowz │ │ │ ├── useContracts.ts // Sorrowz │ │ │ ├── useMarket.ts //
Powerz │ │ │ ├── useProviders.ts // Powerz │ │ │ └── useWebSocket.ts //
Powerz │ │ └── ... │ └── package.json │ ├── docker-compose.yml //
PostgreSQL + Redis ├── turbo.json // Turborepo config └── package.json
```

Technology Standards

LAYER	TECHNOLOGY	VERSION
Smart Contracts	Foundry (forge) + Solidity	^0.8.24

LAYER	TECHNOLOGY	VERSION
Backend Framework	NestJS with Fastify adapter	^10.x
Database	PostgreSQL + Prisma ORM	^16 / ^5.x
Cache / PubSub	Redis (via Upstash or local)	^7
Blockchain RPC	viem (backend) + wagmi (frontend)	^2.x
WebSocket	Socket.io or @nestjs/websockets	^4.x
Testing	forge test (contracts) + Jest (backend)	—
Deployment Target	Base Sepolia (testnet) → Base (mainnet)	—
CI/CD	GitHub Actions	—

Coding Standards

- **TypeScript strict mode** everywhere — no `any` types
- **Prettier + ESLint** shared config in monorepo root
- **Conventional Commits** format: `feat(orders): add batch validation`
- **Branch naming**: `sorrowz/feature-name` or `powerz/feature-name`
- **PR required** for merging to `main` — other dev reviews
- **Solidity**: NatSpec comments on all public functions, OpenZeppelin where possible
- **All API endpoints** must have request/response TypeScript types defined in `shared/`

Environment Variables (Both Devs)

```
# .env.example — both devs must have these
DATABASE_URL="postgresql://user:pass@localhost:5432/darkpool"
REDIS_URL="redis://localhost:6379"
```

```
BASE_SEPOLIA_RPC="https://sepolia.base.org" DEPLOYER_PRIVATE_KEY="0x..."
USDC_CONTRACT_ADDRESS="0x036CbD53842c5426634e7929541eC2318f3dCF7e"
JWT_SECRET="..." API_PORT="3001" WS_PORT="3002"
```

SECTION 03 —

Smart Contracts: Core Trading

These three contracts form the heart of the dark pool protocol — handling order submission, escrow management, and batch settlement verification.

DarkPool.sol

CRITICAL PATH

Main entry point for the protocol. Accepts encrypted order commitments, manages batch lifecycle, and coordinates settlement.

FUNCTION	DESCRIPTION	ACCESS
`submitOrder(bytes32 commitment, uint256 escrowAmount)`	Accept an encrypted order commitment + lock escrow deposit. Emit `OrderSubmitted` event.	Public
`cancelOrder(bytes32 orderId)`	Cancel an unfilled order and release escrowed funds back to the user.	Order owner
`settleBatch(bytes32[] matchedOrders, uint256 clearingPrice, bytes proof)`	Called by the relayer after matching. Verifies proof via SettlementVerifier, then triggers Escrow transfers.	Relayer only
`getCurrentBatchId()`	Returns current batch number (increments each settlement).	View
`getOrderStatus(bytes32 orderId)`	Returns order status: PENDING, FILLED, CANCELLED, EXPIRED.	View

```
// User's browser computes this BEFORE sending to contract: bytes32
commitment = keccak256(abi.encodePacked( gpuType, // e.g., "H100" quantity,
// e.g., 4 pricePerHour, // e.g., 0.25 USDC (in wei) duration, // e.g., 24
hours side, // BUY or SELL secret // random 32 bytes (user keeps this) ));
// Only the commitment hash goes on-chain — order details stay private //
The secret is sent encrypted to the matching engine off-chain
```

- `OrderSubmitted(bytes32 indexed orderId, address indexed user, uint256 escrowAmount)`
- `OrderCancelled(bytes32 indexed orderId, address indexed user)`
- `BatchSettled(uint256 indexed batchId, uint256 clearingPrice, uint256 matchedVolume)`
- `OrderFilled(bytes32 indexed orderId, uint256 clearingPrice)`

> **Security requirements:** Use OpenZeppelin's ReentrancyGuard on all fund-moving functions. Implement Pausable for emergency stops. Use AccessControl for relayer permissions.

Escrow.sol

CRITICAL PATH

Holds buyer USDC funds securely. Funds are locked when an order is placed, released to the seller on successful delivery, and returned to the buyer on cancellation.

FUNCTION	DESCRIPTION
`deposit(uint256 amount)`	User deposits USDC into their escrow balance. Uses IERC20 transferFrom.
`withdraw(uint256 amount)`	User withdraws unlocked USDC from escrow.

FUNCTION	DESCRIPTION
`lockFunds(address user, uint256 amount, bytes32 orderId)`	Called by DarkPool.sol when order is submitted. Moves funds from available to locked.
`releaseFunds(address from, address to, uint256 amount, uint256 fee)`	Called during settlement. Transfers locked funds to seller, fee to FeeCollector.
`unlockFunds(address user, uint256 amount, bytes32 orderId)`	Called on cancellation. Returns locked funds to available balance.
`getBalance(address user)`	Returns (available, locked) balances.

Integration point: Escrow.sol must accept calls from DarkPool.sol (via AccessControl). It interacts with Powerz's FeeCollector.sol for fee deduction during settlement.

SettlementVerifier.sol

Verifies that the off-chain matching engine produced a correct result. For V1, this uses a trusted relayer pattern. In V2, this will verify ZK proofs.

```
// V1: Simple relayer verification // The relayer is a trusted address that submits settlement results // Verification = signature check from authorized relayer function verifySettlement( bytes32[] matchedBuyOrders, bytes32[] matchedSellOrders, uint256 clearingPrice, bytes relayerSignature ) external view returns (bool) { // Verify the relayer signed this settlement data // In V2: replace with ZK proof verification (Noir UltraHonk verifier) }
```

- Replace relayer signature check with on-chain ZK proof verification

- Use Noir (Aztec) to write the matching circuit
- The circuit proves: clearing price maximizes matched volume AND all matched buyers bid ≥ clearing price AND all matched sellers asked ≤ clearing price
- Generate a Solidity verifier from the Noir circuit using `nargo codegen-verifier`

Testing Requirements

All contracts must have comprehensive Foundry tests. Minimum coverage: 90%.

TEST CATEGORY	EXAMPLES
Happy path	Submit order → batch settles → funds transfer correctly
Edge cases	Cancel already-filled order (should revert), withdraw more than available (should revert)
Access control	Non-relayer calling settleBatch (should revert), non-owner calling cancel (should revert)
Reentrancy	Attempt reentrancy on withdraw, releaseFunds
Fuzz tests	Random order amounts, random batch sizes, edge clearing prices
Integration	Full flow: deposit → submit → settle → verify balances changed correctly

SECTION 04 —

Backend Services: Trading Core

Three NestJS modules that handle the most complex business logic in the system — order management, the batch auction matching engine, and on-chain settlement.

Orders Module

CRITICAL PATH

Handles order submission, validation, status tracking, and cancellation. This module receives encrypted order data from users and stores it for the matching engine.

METHOD	ENDPOINT	DESCRIPTION
POST	`/api/v1/orders`	Submit a new encrypted order. Validate commitment, check escrow balance, forward to blockchain.
GET	`/api/v1/orders`	List user's orders with filtering (status, side, GPU type). Paginated.
GET	`/api/v1/orders/:id`	Get single order details + status.
DELETE	`/api/v1/orders/:id`	Cancel an active order. Triggers on-chain cancellation + escrow unlock.

```
prisma schema — orders
id                                  String @id @default(uuid())
userId                              String (FK → users)
commitmentHash                      String @unique
```

```
side                                                    Enum (BUY, SELL)
gpuType                                   String (H100, A100, RTX4090)
quantity                                                            Int
pricePerHour                                                    Decimal
duration                                                   Int (hours)
escrowAmount                                                    Decimal
status                 Enum (PENDING, ACTIVE, FILLED, CANCELLED, EXPIRED)
batchId                                       Int? (null until settled)
clearingPrice                               Decimal? (null until filled)
txHash                                          String? (on-chain tx)
createdAt                                    DateTime @default(now())
updatedAt                                         DateTime @updatedAt
```

- User must be authenticated (API key or wallet signature)
- User must have sufficient available escrow balance ≥ `quantity × pricePerHour × duration`
- GPU type must exist in TokenRegistry (call Powerz's contract)
- Quantity must be ≥ 1 and ≤ 1000
- Price must be within reasonable bounds (0.01 – 100 USDC/hr)
- Duration must be 1 – 720 hours (30 days max)

Matching Engine

MOST COMPLEX MODULE

The core algorithm that runs every 30–60 seconds, collecting all pending orders and finding the clearing price that maximizes matched volume.

```
// Pseudo-code for the clearing price algorithm // Runs as a NestJS cron
job every 30-60 seconds function runBatchAuction(pendingOrders: Order[]):
```

```
BatchResult { // 1. Separate into buy orders and sell orders, grouped by
GPU type const buyOrders = orders.filter(o => o.side ===
'BUY').sort(byPriceDesc) const sellOrders = orders.filter(o => o.side ===
'SELL').sort(byPriceAsc) // 2. Find clearing price: the price where
cumulative buy volume // meets cumulative sell volume (supply = demand
intersection) for each candidate price P: buyVolume = sum(qty) where
buyOrders.price >= P sellVolume = sum(qty) where sellOrders.price <= P
matchedVolume = min(buyVolume, sellVolume) clearingPrice = P that maximizes
matchedVolume // 3. Match orders at clearing price // - All buyers who bid
>= clearingPrice get filled // - All sellers who asked <= clearingPrice get
filled // - Everyone pays/receives the SAME clearing price (uniform
pricing) // 4. Submit settlement on-chain via SettlementVerifier // 5. Emit
events via Powerz's WebSocket server }
```

- **Cron schedule:** Use `@nestjs/schedule` with a configurable interval (default 45 seconds)
- **GPU-type separation:** Run independent auctions per GPU type (H100 auction, A100 auction, etc.)
- **Minimum batch size:** Skip settlement if fewer than 2 orders (1 buy + 1 sell minimum)
- **Redis locking:** Use Redis distributed lock to prevent concurrent batch runs
- **Partial fills:** V1 supports all-or-nothing fills only. Partial fills in V2.
- **Telemetry:** Log batch results (matched volume, clearing price, order count) to database

prisma schema — settlements

id	String @id @default(uuid())
batchId	Int @unique
gpuType	String
clearingPrice	Decimal
matchedVolume	Int (GPU-hours)
buyOrderCount	Int
sellOrderCount	Int
totalValueUSDC	Decimal
protocolFee	Decimal

txHash	String
proofHash	String?
settledAt	DateTime

Settlement Service

Bridges the off-chain matching engine with on-chain settlement. After the matching engine determines results, this service submits the transaction to Base.

- Construct the settlement transaction with matched order IDs, clearing price, and proof
- Submit to DarkPool.sol's `settleBatch()` via viem
- Wait for transaction confirmation (Base block time ~2s)
- Update order statuses in PostgreSQL to FILLED
- Publish settlement event to Redis (Powerz's WebSocket server picks this up)
- Handle failed transactions with retry logic (max 3 retries with exponential backoff)

prisma schema — compute_jobs

id	String @id @default(uuid())
buyOrderId	String (FK → orders)
sellOrderId	String (FK → orders)
providerId	String (FK → providers)
gpuType	String
quantity	Int
duration	Int
pricePerHour	Decimal (clearing price)
totalCost	Decimal
status	Enum (PROVISIONING, ACTIVE, COMPLETED, FAILED)

```
accessEndpoint                    String? (encrypted SSH/API)
startedAt                                           DateTime?
completedAt                                         DateTime?
```

SECTION 05 —

Frontend Integration: Order Flow

Wire the existing React frontend to real smart contracts and the orders backend API.

Scope of Work

- **Install wagmi + viem + @tanstack/react-query** in the frontend package
- **Create contract hooks** — `useDeposit()`, `useSubmitOrder()`, `useCancelOrder()`, `useEscrowBalance()`
- **Create API hooks** — `useOrders()`, `useOrderDetail()`, `useSubmitOrderAPI()`
- **Wire ProductDetail.tsx** — Replace mock order form with real submission (commit-reveal on client side)
- **Wire Orders.tsx** — Replace hardcoded orders with API data + real-time status updates
- **Wire Dashboard.tsx** — Real escrow balance from contract, real order history from API
- **Commitment generation** — Client-side keccak256 hash of order params + random secret

Key Files to Modify

FILE	CHANGES
`src/pages/ProductDetail.tsx`	Wire buy/sell form to real API + contract. Generate commitment, call deposit + submitOrder.
`src/pages/Orders.tsx`	Replace mock data with React Query hook. Add cancel button with contract call.
`src/pages/Dashboard.tsx`	Real escrow balance, real active orders count, real portfolio value.

FILE	CHANGES
`src/hooks/useContracts.ts`	**New file.** wagmi hooks for DarkPool + Escrow contract interactions.
`src/hooks/useOrders.ts`	**New file.** React Query hooks for orders API.
`src/lib/commitment.ts`	**New file.** Client-side commitment hash generation.
`src/config/contracts.ts`	**New file.** Contract addresses + ABIs for Base Sepolia.

SECTION 06 —

Smart Contracts: Token & Fees

Three contracts that handle token management, compute credits, and protocol fee collection/distribution.

TokenRegistry.sol

Manages whitelisted payment tokens (USDC) and GPU type definitions. Acts as the source of truth for what can be traded on the platform.

FUNCTION	DESCRIPTION
`addToken(address token, string symbol)`	Whitelist a new payment token (initially just USDC).
`removeToken(address token)`	De-list a payment token.
`isTokenWhitelisted(address token)`	Check if a token is accepted for payment.
`addGpuType(string name, uint256 minPrice, uint256 maxPrice, uint256 benchmarkScore)`	Register a GPU type (H100, A100, RTX4090) with pricing bounds.
`getGpuType(string name)`	Return GPU type details.
`getAllGpuTypes()`	List all registered GPU types.

GPU	MIN PRICE	MAX PRICE	VRAM
NVIDIA H100	$0.10/hr	$10.00/hr	80 GB
NVIDIA A100	$0.08/hr	$5.00/hr	80 GB
NVIDIA RTX 4090	$0.05/hr	$2.00/hr	24 GB
NVIDIA L40S	$0.06/hr	$3.00/hr	48 GB

ComputeCredit.sol

ERC-20 token representing standardized GPU-hours. Minted when providers deposit compute capacity, burned when compute is consumed.

FUNCTION	DESCRIPTION
`mint(address to, uint256 amount)`	Mint credits when a provider registers compute capacity. Only callable by protocol.
`burn(uint256 amount)`	Burn credits when compute is consumed (job completed).
`burnFrom(address from, uint256 amount)`	Protocol burns credits on behalf of a user after job delivery.

Legal note: This token must be structured as a pure utility token. It is consumed (burned) when used. Never reference it as an investment. Name: "DarkPool Compute Credit" (DPC). Follow SEC no-action letter guidance.

- Extend OpenZeppelin `ERC20`, `ERC20Burnable`, `AccessControl`
- MINTER_ROLE: assigned to the protocol (DarkPool.sol)
- Token has 18 decimals. 1 DPC = 1 GPU-hour (standardized across GPU types)

- Non-transferable in V1 (soulbound) — can be made transferable in V2 if needed

FeeCollector.sol

Collects protocol fees from settlements and distributes them. Receives USDC from Escrow.sol during each batch settlement.

FUNCTION	DESCRIPTION
`collectFee(uint256 amount, uint256 batchId)`	Called by Escrow.sol during settlement. Records fee collection.
`setFeeRate(uint256 buyerFeeBps, uint256 sellerFeeBps)`	Set fee rates in basis points. Default: buyer 80bps (0.8%), seller 20bps (0.2%).
`getFeeRate()`	Returns current (buyerFeeBps, sellerFeeBps).
`withdraw(address to, uint256 amount)`	Owner withdraws collected fees.
`totalCollected()`	View total fees collected to date.

Integration point: FeeCollector must accept calls from Sorrowz's Escrow.sol. Coordinate on the interface — Escrow calls `collectFee()` during `releaseFunds()`.

Testing Requirements

- Token whitelist/delist flows, GPU type registration/retrieval
- Compute credit mint/burn lifecycle, access control on minting
- Fee collection accuracy at various rates (0%, 0.5%, 1%, edge cases)
- Fee withdrawal by owner only

- Integration: fee deduction during a full settlement cycle (coordinate test with Sorrowz's contracts)

Deployment Script

Powerz owns the deployment script for all 6 contracts (including Sorrowz's). Deploy order matters due to inter-contract references.

```
// Deploy order: // 1. TokenRegistry.sol (standalone) // 2. ComputeCredit.sol (standalone) // 3. FeeCollector.sol (standalone) // 4. Escrow.sol (needs FeeCollector address) // 5. SettlementVerifier.sol (standalone) // 6. DarkPool.sol (needs Escrow + SettlementVerifier + TokenRegistry addresses) // // Post-deploy: // - Grant RELAYER_ROLE on DarkPool to the backend relayer wallet // - Grant DARKPOOL_ROLE on Escrow to DarkPool contract // - Grant MINTER_ROLE on ComputeCredit to DarkPool contract // - Register initial GPU types on TokenRegistry // - Whitelist USDC on TokenRegistry
```

SECTION 07 —

Backend Services: Infrastructure

Five NestJS modules that handle authentication, provider management, market data, real-time communications, and blockchain event indexing.

Auth Module

CRITICAL PATH

Handles wallet-based authentication and API key management. Every other module depends on this for user identity.

METHOD	ENDPOINT	DESCRIPTION
POST	`/api/v1/auth/nonce`	Generate a random nonce for wallet signature (SIWE standard).
POST	`/api/v1/auth/verify`	Verify wallet signature, create/return user, issue JWT.
POST	`/api/v1/auth/api-key`	Generate API key pair (live + test). Requires authenticated session.
DELETE	`/api/v1/auth/api-key/:id`	Revoke an API key.
GET	`/api/v1/auth/me`	Return current user profile.

```
// Sign-In With Ethereum (SIWE) flow: // 1. Frontend calls GET /auth/nonce
→ receives random nonce // 2. Frontend asks wallet to sign message: "Sign
in to DarkPool\nNonce: {nonce}" // 3. Frontend sends POST /auth/verify {
address, signature, nonce } // 4. Backend verifies signature using viem's
```

```
verifyMessage() // 5. Backend creates user if new, issues JWT (24h expiry)
// 6. All subsequent requests include: Authorization: Bearer {jwt} // //
API Key flow (for AI agents): // 1. Authenticated user calls POST
/auth/api-key // 2. Backend generates: sk_live_... and sk_test_... // 3.
Store hashed key in DB. Return raw key ONCE. // 4. Agent sends: X-API-Key:
sk_live_... with each request
```

prisma schema — users

id	String @id @default(uuid())
walletAddress	String @unique
subscriptionTier	Enum (FREE, PRO, ENTERPRISE)
createdAt	DateTime

prisma schema — api_keys

id	String @id @default(uuid())
userId	String (FK → users)
keyHash	String (bcrypt hash of key)
prefix	String (sk_live_ or sk_test_)
environment	Enum (LIVE, TEST)
lastUsedAt	DateTime?
revokedAt	DateTime?
createdAt	DateTime

- `JwtAuthGuard` — validates JWT from Authorization header
- `ApiKeyAuthGuard` — validates API key from X-API-Key header
- `CombinedAuthGuard` — accepts either JWT or API key (used by most endpoints)
- Export a `@CurrentUser()` decorator that injects the authenticated user into route handlers

Critical: All of Sorrowz's endpoints depend on these guards. Export them from the auth module so Sorrowz can import and use `@UseGuards(CombinedAuthGuard)` on order routes.

Provider Module

GPU provider registration, benchmarking, reputation tracking, and listing management.

METHOD	ENDPOINT	DESCRIPTION
POST	`/api/v1/providers`	Register as a GPU provider. Submit hardware specs.
GET	`/api/v1/providers/me`	Get own provider profile, listings, earnings.
PUT	`/api/v1/providers/me`	Update provider profile/specs.
POST	`/api/v1/providers/listings`	Create a new GPU listing (available hours, min price).
GET	`/api/v1/providers/listings`	List active GPU offerings.
DELETE	`/api/v1/providers/listings/:id`	Remove a listing.
GET	`/api/v1/providers/earnings`	Earnings history + payout records.

```
prisma schema — providers

id                                  String @id @default(uuid())
userId                              String @unique (FK → users)
gpuModel                                                String
gpuCount                                                   Int
```

vram	Int (GB)
bandwidth	Int (Gbps)
location	String (region)
benchmarkScore	Float?
reputationScore	Float @default(0)
totalEarnings	Decimal @default(0)
stakedAmount	Decimal @default(0)
uptime	Float @default(0) (percentage)
status	Enum (PENDING, ACTIVE, SUSPENDED)
createdAt	DateTime

Market Module

Aggregated, anonymized market statistics and analytics. Powers the Analytics page and public market data.

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/v1/market/stats`	Overall stats: 24h volume, total providers, total GPU-hours, avg clearing prices.
GET	`/api/v1/market/depth`	Anonymized depth: aggregated buy/sell pressure per GPU type (no individual orders).
GET	`/api/v1/market/price-history`	Historical clearing prices by GPU type. Params: gpuType, timeframe (1h, 24h, 7d, 30d).
GET	`/api/v1/market/settlements`	Recent batch settlements (public data: batch ID, clearing price, volume, timestamp).

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/v1/account/balance`	User's escrow balance (reads from Escrow contract via viem).

- Query `settlements` table (written by Sorrowz's matching engine) for price history
- Query `orders` table for aggregated depth (count pending buys/sells per GPU type at price bands)
- Cache frequently-accessed stats in Redis with 15-second TTL
- Read escrow balance directly from blockchain via viem `readContract`

WebSocket Server

Pushes real-time updates to connected frontends and API consumers. Subscribes to Redis pub/sub channels for events.

EVENT	PAYLOAD	WHO RECEIVES
`batch:settled`	{ batchId, gpuType, clearingPrice, matchedVolume, timestamp }	All connected clients
`order:filled`	{ orderId, clearingPrice, batchId }	Order owner only
`order:cancelled`	{ orderId }	Order owner only
`market:update`	{ gpuType, currentPrice, volume24h }	All connected clients
`provider:matched`	{ orderId, buyerDetails (encrypted) }	Provider only

- Use `@nestjs/websockets` with Socket.io adapter
- Subscribe to Redis pub/sub channel `darkpool:events`

- Sorrowz's settlement service publishes events to this channel after each batch
- Authenticate WebSocket connections using JWT (sent in handshake query params)
- Room-based routing: `user:{userId}` for private events, `market` for public

Event Indexer

Listens to smart contract events on Base and syncs on-chain state to the PostgreSQL database.

- `OrderSubmitted` → Update order status to ACTIVE in DB
- `OrderCancelled` → Update order status to CANCELLED in DB
- `BatchSettled` → Create settlement record in DB, update matched orders to FILLED
- `OrderFilled` → Update individual order with clearing price and batch ID
- `Transfer` (USDC) → Track deposit/withdrawal events for balance history

- Use viem's `watchContractEvent` for real-time listening
- Implement a block cursor (store last processed block in Redis) for crash recovery
- On startup, backfill any missed blocks since last cursor
- Publish indexed events to Redis pub/sub for WebSocket distribution

SECTION 08 —

Frontend Integration: Market & Real-Time

Wire the existing React frontend to market data, provider management, and real-time WebSocket feeds.

Scope of Work

- **Create API hooks** — `useMarketStats()`, `usePriceHistory()`, `useMarketDepth()`
- **Create provider hooks** — `useProviderProfile()`, `useProviderListings()`, `useProviderEarnings()`
- **Create WebSocket hook** — `useWebSocket()` with auto-reconnect and event subscription
- **Wire Marketplace.tsx** — Real GPU listings from API instead of hardcoded 24 products
- **Wire Analytics.tsx** — Real price history charts, real market stats
- **Wire Provider.tsx** — Real registration form, real earnings data
- **Wire DashboardSidebar.tsx** — Real-time batch count in live feed
- **Wire SettingsPage.tsx** — Real API key generation/revocation via auth endpoints
- **Wire LiveFeed component** — Real-time settlement feed from WebSocket
- **Wire AuctionTimer** — Sync countdown with actual backend batch cycle

Key Files to Modify

FILE	CHANGES
`src/pages/Marketplace.tsx`	Replace 24 hardcoded products with API data from /market/stats + /providers/listings.

FILE	CHANGES
`src/pages/Analytics.tsx`	Replace mock chart data with /market/price-history, /market/stats.
`src/pages/Provider.tsx`	Wire registration form to POST /providers, wire earnings to GET /providers/earnings.
`src/pages/SettingsPage.tsx`	Wire API key CRUD to /auth/api-key endpoints.
`src/components/dashboard/LiveFeed.tsx`	Replace mock feed with WebSocket batch:settled events.
`src/components/dashboard/AuctionTimer.tsx`	Sync with backend batch cycle timing.
`src/hooks/useMarket.ts`	**New file.** React Query hooks for market endpoints.
`src/hooks/useProviders.ts`	**New file.** React Query hooks for provider endpoints.
`src/hooks/useWebSocket.ts`	**New file.** Socket.io client with auto-reconnect + event typing.
`src/lib/api.ts`	**New file.** Axios/fetch instance with auth interceptor (JWT + API key).

SECTION 09

Shared Interfaces & Data Contracts

Both developers must agree on these TypeScript interfaces. They live in `packages/backend/src/shared/types/` and are the contract between modules.

Order Types

```
// shared/types/order.types.ts export enum OrderSide { BUY = 'BUY', SELL = 'SELL' } export enum OrderStatus { PENDING = 'PENDING', // Submitted to backend, not yet on-chain ACTIVE = 'ACTIVE', // Confirmed on-chain, waiting for batch FILLED = 'FILLED', // Matched in a batch auction CANCELLED = 'CANCELLED', // Cancelled by user EXPIRED = 'EXPIRED', // Passed expiry time without fill } export enum GpuType { H100 = 'H100', A100 = 'A100', RTX4090 = 'RTX4090', L40S = 'L40S' } export interface CreateOrderDto { side: OrderSide; gpuType: GpuType; quantity: number; pricePerHour: number; // in USDC (e.g., 0.25) duration: number; // in hours commitmentHash: string; // keccak256 from client encryptedDetails: string; // encrypted order params for matching engine } export interface OrderResponse { id: string; side: OrderSide; gpuType: GpuType; quantity: number; pricePerHour: number; duration: number; escrowAmount: number; status: OrderStatus; batchId?: number; clearingPrice?: number; txHash?: string; createdAt: string; }
```

Settlement Types

```
// shared/types/settlement.types.ts export interface BatchResult { batchId: number; gpuType: GpuType; clearingPrice: number; matchedBuyOrders: string[]; // order IDs matchedSellOrders: string[]; // order IDs matchedVolume: number; // total GPU-hours totalValueUSDC: number;
```

```
protocolFee: number; } // Published to Redis channel "darkpool:events" by
Sorrowz's settlement service // Consumed by Powerz's WebSocket server
export interface SettlementEvent { type: 'batch:settled' | 'order:filled' |
'order:cancelled'; payload: BatchResult | { orderId: string;
clearingPrice?: number }; timestamp: number; }
```

Market & Provider Types

```
// shared/types/market.types.ts export interface MarketStats {
totalVolume24h: number; totalProviders: number; totalGpuHours: number;
avgClearingPrices: Record<GpuType, number>; batchCount24h: number; } export
interface PriceHistoryPoint { timestamp: string; price: number; volume:
number; } // shared/types/provider.types.ts export interface
RegisterProviderDto { gpuModel: GpuType; gpuCount: number; vram: number;
bandwidth: number; location: string; } export interface ProviderResponse {
id: string; gpuModel: GpuType; gpuCount: number; reputationScore: number;
totalEarnings: number; uptime: number; status: 'PENDING' | 'ACTIVE' |
'SUSPENDED'; }
```

Integration protocol: When either developer needs to publish or consume data from the other's domain, they use Redis pub/sub with the `SettlementEvent` interface. Never call another dev's module functions directly — use the shared event bus.

SECTION 10

Timeline & Milestones

Both developers work in parallel. Integration checkpoints ensure the pieces fit together.

WEEK 1–2

Foundation

Sorrowz: DarkPool.sol + Escrow.sol + SettlementVerifier.sol with full test suite. Deploy to Base Sepolia.

Powerz: NestJS project setup + Auth module + TokenRegistry.sol + ComputeCredit.sol + FeeCollector.sol. Deploy to Base Sepolia.

INTEGRATION: DEPLOY ALL 6 CONTRACTS TOGETHER

WEEK 3–4

Core Backend

Sorrowz: Orders module + Matching engine (batch auction algorithm). Unit tests for matching logic.

Powerz: Provider module + Market module + Database migrations. Set up Redis.

INTEGRATION: TEST ORDER FLOW END-TO-END

WEEK 5–6

Real-Time & Settlement

Sorrowz: Settlement service (on-chain submission). Connect matching engine → settlement → Redis events.

Powerz: WebSocket server + Event indexer. Subscribe to Redis events, push to clients.

INTEGRATION: FULL CYCLE TEST (ORDER → MATCH → SETTLE → WS NOTIFICATION)

WEEK 7–8

Frontend Integration

Sorrowz: Wire order flow (ProductDetail, Orders, Dashboard) to real API + contracts. Client-side commitment generation.

Powerz: Wire market data, provider pages, settings, live feed, auction timer to real API + WebSocket.

INTEGRATION: FULL UI WALKTHROUGH ON TESTNET

WEEK 9–10

Testing & Polish

Both: End-to-end testing, bug fixes, error handling, loading states, edge cases. Prepare for testnet launch.

JOINT: E2E TESTING + BUG BASH

SECTION 11

Dependency Map & Integration Points

Where the two developers' work touches. These are the critical handoff points that require coordination.

```
                              SYSTEM ARCHITECTURE

                        Frontend (React + Vite)

     Order Flow (Sorrowz)          Market + Provider (Powerz)
     ProductDetail.tsx             Marketplace.tsx
     Orders.tsx                    Analytics.tsx
     Deepbook.tsx                  Provider.tsx
     useContracts.ts               useWebSocket.ts
     useOrders.ts                  useMarket.ts

                                   | HTTP + WebSocket

                        NestJS Backend

          ┌──────────────┐   Redis   ┌──────────────┐
| Orders      |──publish──>| darkpool:    |──subscribe──>| WebSocket    |
| Matching    |            | events       |              | Server       |
| Settlement  |            └──────────────┘              | Indexer      |
└──────┬──────┘                                        | Auth         |
       |                                               | Providers    |
                  PostgreSQL                           | Market       |
                  | orders     |                       └──────┬───────┘
                  | settlement |
                  | compute_gas|
                  | users      |
```

```
Base Blockchain

DarkPool.sol  Escrow.sol  SettlementVerifier.sol
TokenRegistry.sol  ComputeCredit.sol  FeeCollector.sol
```

Critical Integration Points

#	INTEGRATION	OWNER	DEPENDS ON	PROTOCOL
1	**Auth guards on order endpoints**			Import `CombinedAuthGuard` from auth module
2	**Settlement → WebSocket events**			Redis pub/sub channel `darkpool:events`
3	**Escrow → FeeCollector**			Solidity: `IFeeCollector.collectFee()`
4	**Orders → TokenRegistry validation**			Solidity: `ITokenRegistry.getGpuType()`
5	**Market module reads settlements**			PostgreSQL: `settlements` table

#	INTEGRATION	OWNER	DEPENDS ON	PROTOCOL
6	**Contract deployment script**		Both contracts	Foundry deploy script with correct ordering

Coordination rule: Before either developer merges changes that affect an integration point, they must notify the other developer and ensure the shared interface hasn't changed. All shared types live in `packages/backend/src/shared/` — changes to this directory require both devs' approval.

Communication Protocol

- **Daily sync:** Brief check-in on what was completed and any blockers at integration points
- **PR reviews:** Each developer reviews the other's PRs before merge to main
- **Integration tests:** At each milestone (end of Week 2, 4, 6, 8), run the full integration test suite together
- **Shared types changes:** Any change to `shared/types/` must be communicated and approved by both
- **Contract interface changes:** If a Solidity function signature changes, both devs must update their dependent code

AGENTIC DARK POOL

Internal Development Document — Confidential

Version 1.0 — March 29, 2026

This document defines the complete work assignment for backend implementation. Both developers should reference the original Research & Build Guide for product context and business requirements.